

Relatório da Análise de Algoritmos de Árvores Geradoras Mínimas(Prim e Kruskal)

Bruna Fontes Castro

Felipe do Amparo Lauria

Instituto de Biociências, Letras e Ciências Exatas, Unesp -
Universidade Estadual Paulista, 15054-000, São José do Rio Preto - SP,
Brazil.

Novembro 2024

1 Introdução

Os algoritmos de Kruskal e Prim desempenham um papel central na teoria dos grafos e em diversas aplicações práticas, sendo amplamente utilizados para determinar a Árvore Geradora Mínima (AGM) em grafos ponderados. A AGM de um grafo conexo é um subgrafo que conecta todos os vértices com o menor custo total possível, garantindo que não haja ciclos. Esses algoritmos são fundamentais em áreas como redes de computadores, planejamento de circuitos elétricos, sistemas de transporte e muitas outras aplicações que envolvem grafos.

Este relatório foca na análise comparativa do desempenho dos algoritmos de Kruskal e Prim em dois tipos distintos de grafos: esparsos, caracterizados por um número reduzido de arestas em relação ao número de vértices, e densos, onde o número de arestas é próximo ao máximo permitido ($|E| \approx |V|^2$). Ambas as categorias são relevantes para problemas reais, sendo que grafos esparsos modelam sistemas como redes de transporte ou cadeias de suprimentos, enquanto grafos densos representam, por exemplo, redes de comunicação completas.

Para conduzir a análise, implementamos ambos os algoritmos e realizamos experimentos variando o tamanho dos grafos de 10 a 10.000 vértices. Os tempos de execução foram registrados, e gráficos foram gerados para visualizar as diferenças de desempenho. Além disso, fornecemos uma descrição teórica das estruturas de dados subjacentes — Union-Find e Fila de Prioridades — que são cruciais para a eficiência dos algoritmos. A análise incluiu a avaliação da complexidade computacional teórica e sua relação com os resultados experimentais obtidos.

Os resultados apresentados não apenas destacam as condições em que cada algoritmo é mais eficiente, mas também demonstram como o comportamento

varia dependendo das características topológicas do grafo. Assim, este relatório contribui para um entendimento mais profundo das nuances dos algoritmos de Kruskal e Prim e suas aplicações em diferentes cenários.

2 Descrição dos Algoritmos

2.1 Union-Find (utilizado no algoritmo de Kruskal)

Union-Find é uma estrutura de dados eficiente utilizada para gerenciar e unir componentes disjuntos, essencial no algoritmo de Kruskal para evitar ciclos durante a construção da Árvore Geradora Mínima (AGM). A estrutura é composta por duas operações principais:

- **Find:** Identifica a raiz do conjunto ao qual um elemento pertence. Para aumentar a eficiência, utiliza a técnica de *path compression*, que comprime o caminho percorrido, conectando diretamente os elementos à raiz.
- **Union:** Une dois conjuntos disjuntos. A operação é otimizada pela técnica de *union by rank*, que conecta a árvore de menor profundidade à raiz da árvore de maior profundidade, reduzindo o tamanho da estrutura.

Com essas otimizações, as operações de *Find* e *Union* possuem complexidade quase constante, $O(\alpha(n))$, onde α é a inversa da função de Ackermann, uma função de crescimento extremamente lento.

No algoritmo de Kruskal, Union-Find é utilizado para verificar se duas extremidades de uma aresta pertencem ao mesmo componente, garantindo que apenas arestas válidas sejam adicionadas à AGM.

2.2 Fila de Prioridades (utilizada no algoritmo de Prim)

A fila de prioridades é uma estrutura de dados que organiza elementos com base em um valor de prioridade, permitindo a extração eficiente do elemento de menor ou maior prioridade. No algoritmo de Prim, essa estrutura é fundamental para selecionar a aresta de menor peso que conecta um vértice visitado a um não visitado.

Implementações comuns de filas de prioridades utilizam *heaps* binários, como *binary heap*, que oferecem as seguintes complexidades:

- Inserção de elementos: $O(\log n)$.
- Extração do elemento de menor prioridade (mínimo): $O(\log n)$.
- Atualização da prioridade de um elemento: $O(\log n)$.

No contexto do algoritmo de Prim, a fila de prioridades é usada para armazenar as arestas candidatas à inclusão na AGM. A cada iteração, a aresta de menor peso é extraída, e as novas arestas que conectam o vértice recém-visitado aos vértices adjacentes são adicionadas à fila, mantendo a eficiência do processo.

Essas estruturas de dados desempenham papéis essenciais na eficiência computacional dos algoritmos, tornando-os viáveis para aplicações práticas em grafos de grande escala.

3 Análise dos Algoritmos

Grafos podem ser classificados com base na densidade de suas arestas. Em termos gerais, um grafo **esparso** possui um número de arestas relativamente pequeno em relação ao número máximo possível ($|E| \approx |V|$), enquanto um grafo **denso** apresenta uma quantidade de arestas próxima ao limite máximo permitido ($|E| \approx |V|^2$). Na prática, grafos esparsos são comuns em redes sociais e sistemas de transporte, enquanto grafos densos aparecem em aplicações como redes de comunicação completas.

3.1 Resultados e Análise para Grafos Esparsos

Os tempos de execução medidos para grafos esparsos variaram de micro a milissegundos conforme o número de vértices aumentou de 10 para 10.000. O `tab:esparso` apresenta os tempos de execução detalhados para os algoritmos de Kruskal e Prim em grafos esparsos, enquanto a `fig:grafico esparsos` ilustra a comparação gráfica desses tempos.

O algoritmo de Kruskal mostrou-se consistentemente mais rápido do que Prim em grafos pequenos e médios devido à eficiência do *Union-Find* no tratamento das arestas. Para grafos maiores, os tempos de execução dos dois algoritmos se aproximaram, sugerindo que a abordagem baseada em filas de prioridade do algoritmo de Prim também é eficiente quando as arestas são limitadas.

Table 1: Tempos de execução para grafos esparsos.

Número de Vértices	Tempo (Kruskal)	Tempo (Prim)
10	0.000000 s	0.000992 s
100	0.000000 s	0.000999 s
500	0.0.001009 s	0.000999 s
1.000	0.002001 s	0.002011 s
2.000	0.001995 s	0.004013 s
5.000	0.007007 s	0.010397 s
10.000	0.019021 s	0.023010 s

Os tempos de execução indicam que ambos os algoritmos são adequados para grafos esparsos, mas Kruskal é preferido quando se deseja explorar a simplicidade e eficiência do processamento direto das arestas. A análise da `tab:esparso` e da `fig:grafico esparsos` reforça a superioridade inicial de Kruskal devido à sua complexidade teórica $O(E \log E)$, especialmente para grafos com número reduzido de arestas.

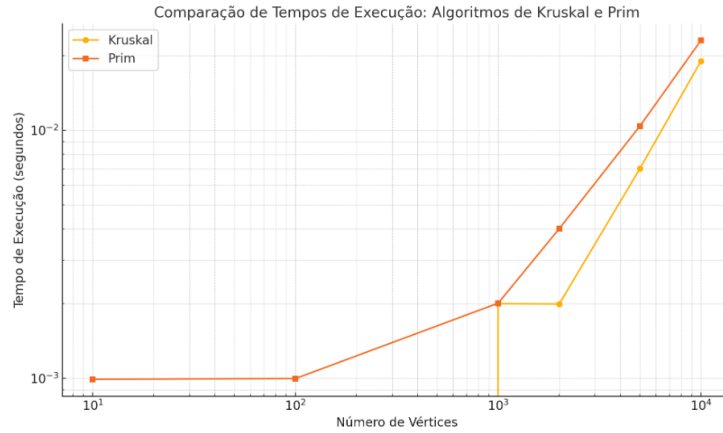


Figure 1: Comparação de tempos de execução dos algoritmos em grafos esparsos.

3.2 Resultados e Análise para Grafos Densos

Nos grafos densos, o número de arestas cresce exponencialmente em relação ao número de vértices. Isso fez com que os tempos de execução aumentassem de maneira significativa. A `tab:denso` apresenta os dados detalhados de execução para os dois algoritmos, enquanto a `fig:grafico denso` mostra a comparação gráfica desses tempos.

O algoritmo de Prim apresentou uma vantagem consistente em grafos densos, mesmo para os maiores grafos analisados (10.000 vértices). Essa eficiência pode ser atribuída ao fato de Prim explorar vértices conectados de forma incremental, minimizando operações redundantes em redes densamente interconectadas.

Table 2: Tempos de execução para grafos densos.

Número de Vértices	Tempo (Kruskal)	Tempo (Prim)
10	0.000000 s	0.001000 s
100	0.002015 s	0.001010 s
500	0.087993 s	0.045009 s
1.000	0.406077 s	0.215059 s
2.000	1.732800 s	0.933900 s
5.000	11.248400 s	6.124400 s
10.000	45.557800 s	24.888800 s

Por outro lado, Kruskal, ao ordenar todas as arestas previamente, experimenta um impacto significativo no tempo de execução, como demonstrado na `tab:denso` e na `fig:grafico denso`. Para grafos densos, a complexidade $O(E \log E)$ domina, levando a tempos muito superiores a $O(E \log V)$ de Prim.

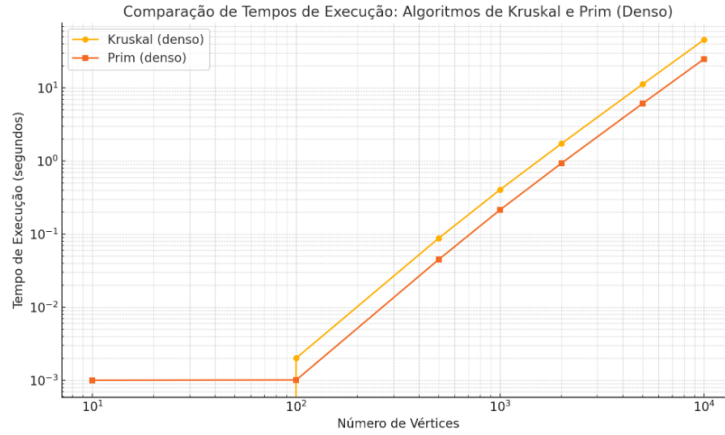


Figure 2: Comparação de tempos de execução dos algoritmos em grafos densos.

4 Conclusão

A análise comparativa dos algoritmos de Kruskal e Prim para a determinação da Árvore Geradora Mínima em grafos revelou diferenças significativas no desempenho, dependendo das características topológicas dos grafos (esparsidade ou densidade).

Para grafos **esparsos**, o algoritmo de Kruskal demonstrou maior eficiência em geral, devido à simplicidade e eficiência da estrutura Union-Find, que processa diretamente as arestas do grafo. Isso é coerente com sua complexidade teórica $O(E \log E)$, que se beneficia do número reduzido de arestas em relação ao número de vértices.

Em contrapartida, para grafos **densos**, o algoritmo de Prim apresentou desempenho superior. Isso se deve à exploração incremental das arestas conectadas aos vértices visitados, otimizando o uso da fila de prioridades. A complexidade $O(E \log V)$ de Prim mostrou-se mais favorável, dado que $E \approx V^2$ em grafos densos, destacando sua eficiência para redes densamente interconectadas.

De forma geral:

- **Kruskal** é recomendado para grafos esparsos, especialmente em cenários onde o número de arestas é limitado em relação ao número de vértices.
- **Prim** é mais eficiente para grafos densos ou quando a manipulação de vértices é mais direta.

Esses resultados reforçam a importância de considerar as características estruturais do problema antes de selecionar o algoritmo a ser utilizado. Ambos os algoritmos possuem papéis cruciais em aplicações práticas, mas sua eficiência depende diretamente da densidade e do tamanho do grafo. Assim, a escolha do algoritmo ideal deve balancear a teoria computacional com os cenários reais de aplicação.